# Laboratório Funcional · Aula 12

Natália Possani de Lima · RA 24190450-2 · Turma ESOFT6S-C
Gerado em 16/10/2026, 19:42:44 · tempo registrado no cronômetro: 0:00

## Preparação: Programar com funções matemáticas

**def area(r): return 3.14159 * r * r**

Pura

**def agora(): return datetime.now()**

Impura

**def total(lista): return sum(lista)**

Pura

**def registra(msg): print(msg)**

Impura

**def sorteia(): return random.randint(1, 6)**

Impura

**def poe(lista, x): lista.append(x); return lista**

Impura

**0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.**

agora() é impura porque depende do horário em que é executada, já que pode retornar resultados diferentes em chamadas distintas

poe(lista, x) é impura porque modifica a lista recebida, acrescentando um elemento. entao, ela altera um dado externo ao resultado retornado pela função.

## Prática 1: Funções puras e formas funcionais

**Previsão: O que as linhas 1, 4 e 7 vão imprimir?**

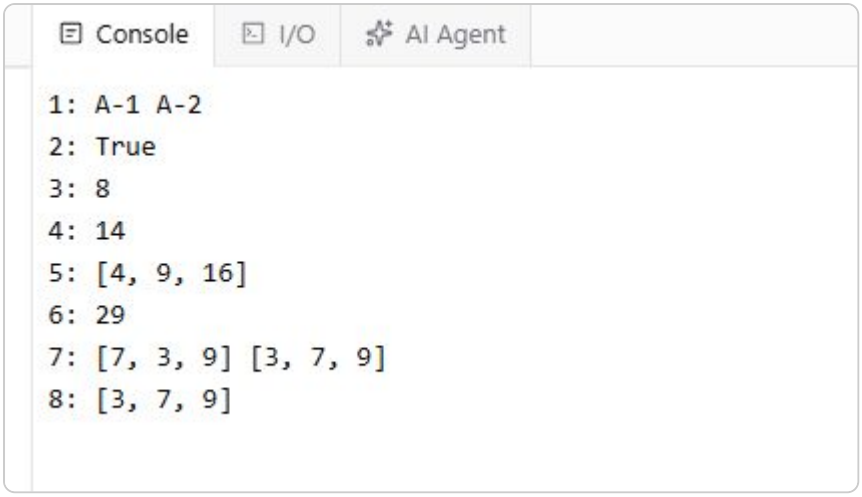
1: A-1 A-2

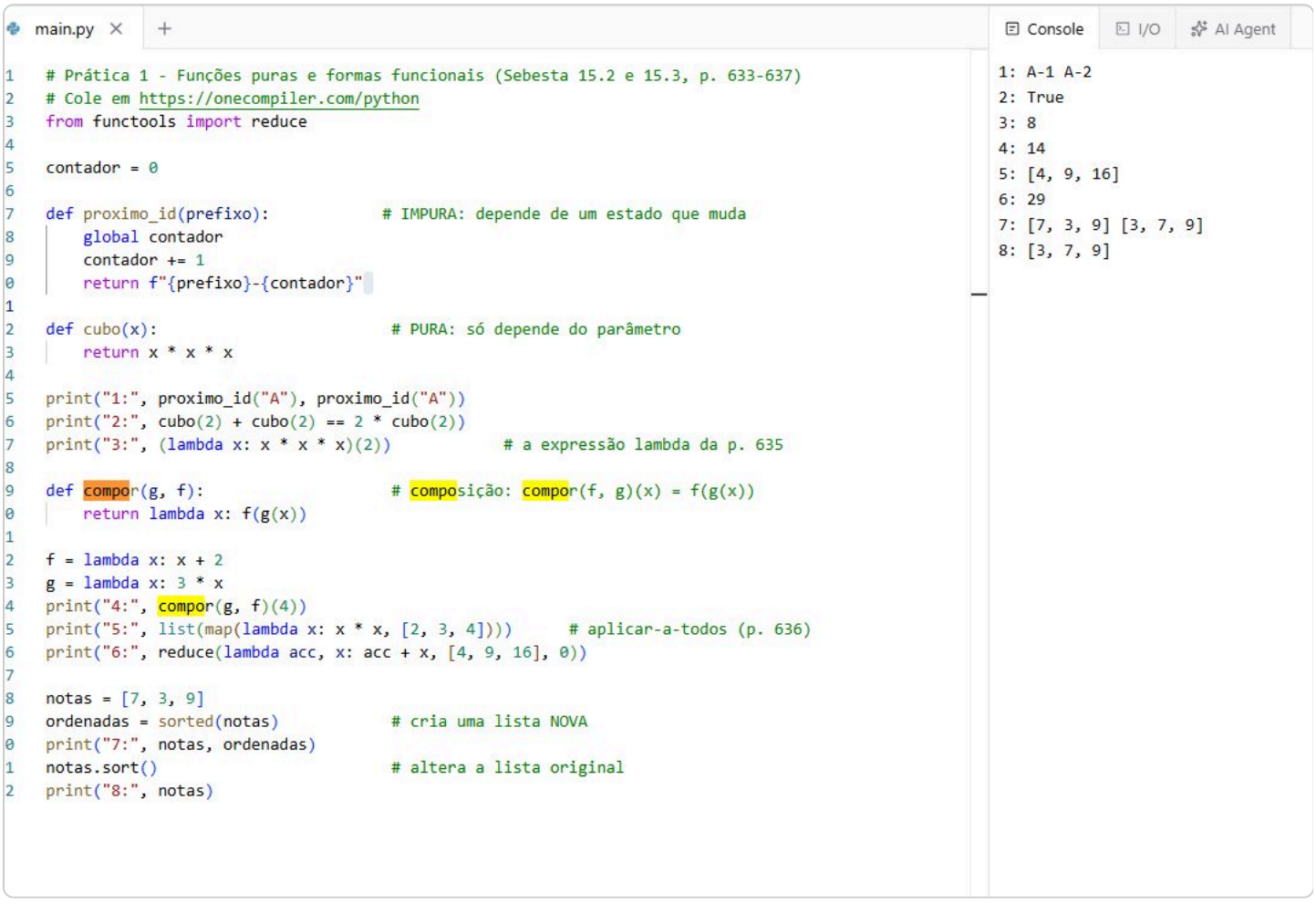
4: 14

7: [7, 3, 9] [3, 7, 9]

**Saídas (original e com compor(g, f))**

*não respondido*




**1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?**

A função devolve valores diferentes porque utiliza uma variável global chamada contador, que é incrementada a cada chamada. Isso dificulta os testes porque o resultado depende do estado anterior do programa, tornando necessário considerar a ordem das chamadas para verificar o comportamento da função.

**1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.**

Na composição compor(f, g)(4), primeiro calculamos (g(4)=3\times4=12) e depois (f(12)=12+2=14)

**1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?**

A função sorted(notas) cria uma nova lista ordenada, mantendo a lista original inalterada. Já notas.sort() ordena a própria lista, modificando-a. Portanto, sorted(notas) segue melhor o estilo funcional, pois evita modificar os dados

recebidos.

## Prática 2: Scheme: tudo é lista

**Previsão: (car '(A B C))**

bc

**Previsão: (cdr '(A B C))**

abc

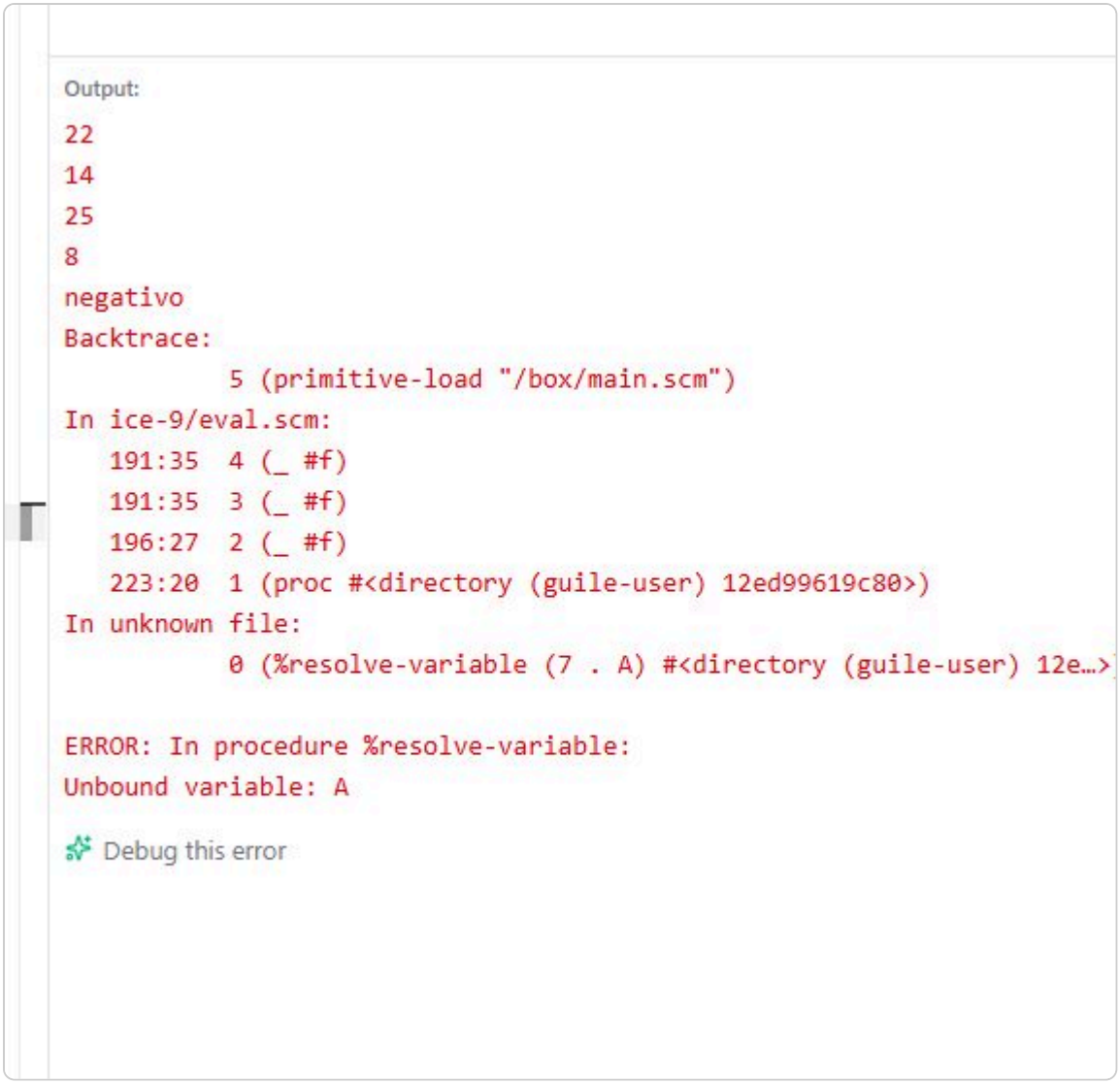
**Previsão: (cons '(A B) '(C D))**

((ab) cd)

**Saídas (original e sem o apóstrofo)**

*não respondido*

```
Output:
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B
```




**2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?**

O apóstrofo serve para indicar que a lista deve ser tratada como dado, sem avaliar seus elementos como uma expressão. Sem o apóstrofo, o Scheme tentou interpretar (A B C) como uma chamada de função, considerando A como o nome de uma função e B e C como argumentos. Como essa chamada não é válida nesse contexto, ocorre um erro.

**2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.**

```
(car (cdr (cdr '(A B C))))
```

## Prática 3: Recursão e recursão em cauda

**O que vi no passo a passo**

*não respondido*

**Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?**

vai funcionar, pois a recursão em cauda permite otimizar as chamadas e reutilizar o espaço da pilha. Já em Python, a função não vai conseguir completar um milhão de chamadas, porque a linguagem não otimiza a recursão em cauda e possui um limite de recursão, gerando um erro RecursionError.

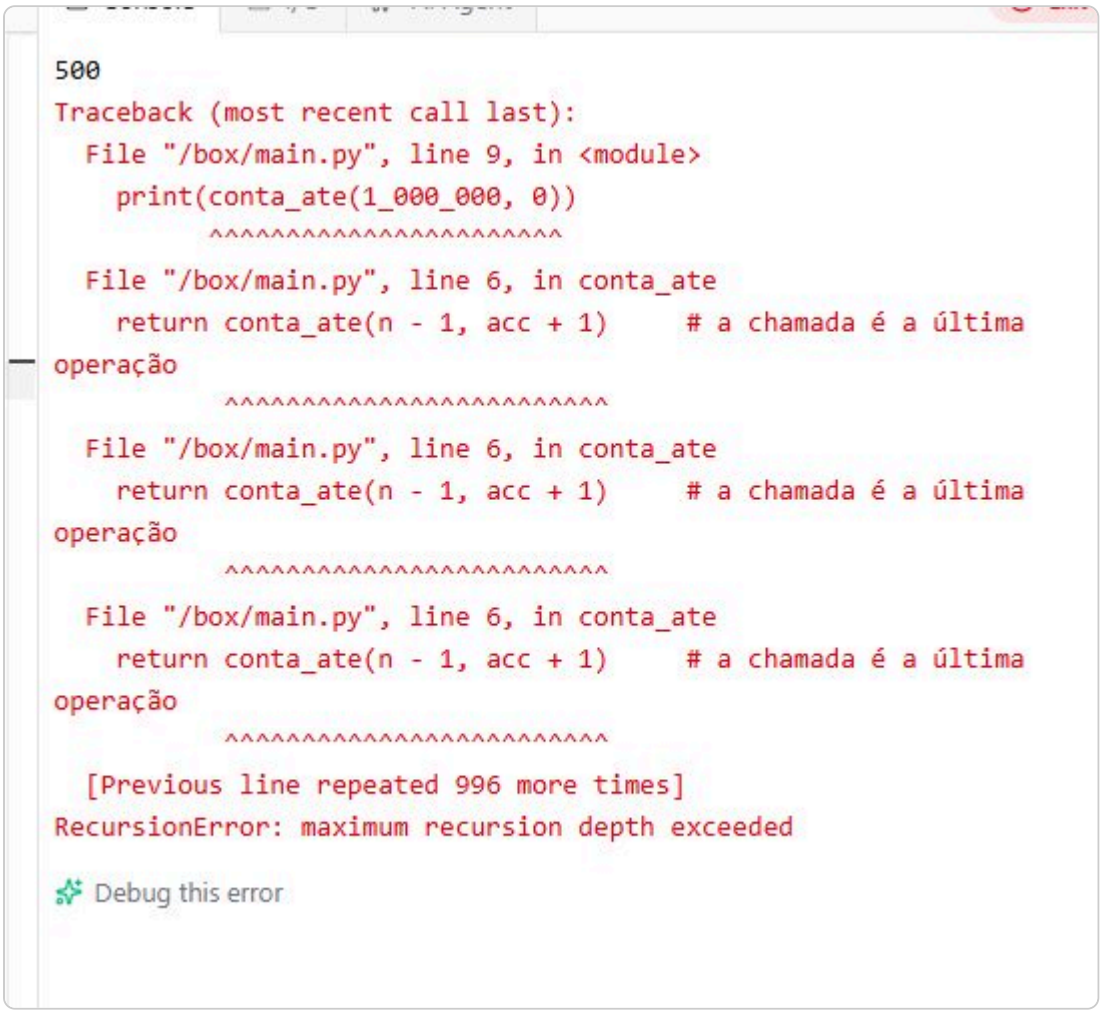
**Saída do Scheme**

*não respondido*

```
Output:
#t
#f
2432902008176640000
2432902008176640000
1000000
```

**Saída do Python (as últimas linhas bastam)**

*não respondido*



```
500
Traceback (most recent call last):
  File "/box/main.py", line 9, in <module>
    print(conta_ate(1_000_000, 0))
          ^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última
operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última
operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última
operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded

Debug this error
```

**3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?**

Na recursão comum, crescem os registros de ativação na pilha, pois cada chamada precisa esperar a chamada seguinte retornar para realizar a multiplicação pendente. Na recursão em cauda, a chamada recursiva é a última operação, não havendo cálculos pendentes após seu retorno. Com a otimização de cauda, a pilha pode reutilizar o registro de ativação.

**3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)**

O Scheme utilizado no exercício consegue otimizar a recursão em cauda, reutilizando o espaço da pilha. Já o Python não realiza essa otimização e possui um limite de profundidade de recursão. Por isso, o programa Python

gera RecursionError antes de completar um milhão de chamadas.

## Prática 4: Mão na massa: três funções sobre listas

**Saída final (com os três ok)**

*não respondido*

```
Output:
tamanho: 0  (esperado 3)
soma-quadrados: 0  (esperado 14)
pares: ()  (esperado (2 4 6))
```

```
três linhas.
```

```
Input for the program ( Optional )

Output:
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

**4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).**

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis))
         (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis))
         (cons (car lis) (pares (cdr lis))))
        (else
         (pares (cdr lis)))))
```

**4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?**

A parte mais difícil foi entender como percorrer a lista recursivamente sem utilizar laços. Usei null? para verificar se a lista estava vazia, car para acessar o primeiro elemento e cdr para continuar a função com o restante da lista. Na função pares, também utilizei even? para identificar os números pares e cons para construir a lista de resultados.

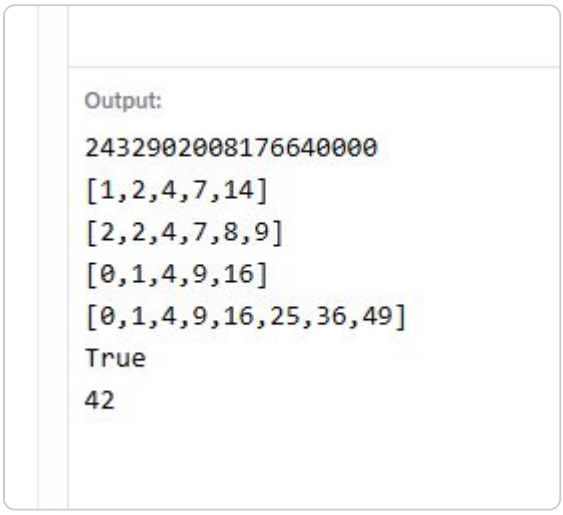
## Prática 5: Haskell: padrões, compreensões e preguiça

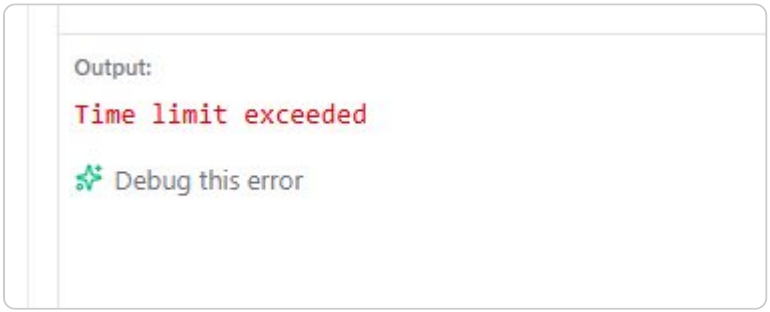
**Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?**

2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42

**Saídas (original e com member 17)**

*não respondido*




**5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?**

O Haskell utiliza avaliação preguiçosa, calculando apenas os valores necessários. Por isso, take 5 squares solicita somente os cinco primeiros elementos da lista infinita. Na expressão primeiro 42 (div 1 0), a função primeiro devolve o primeiro argumento e não utiliza o segundo. Assim, a divisão por zero não precisa ser calculada e nenhum erro acontece.

**5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)**

A busca não termina porque a lista squares é infinita e seus elementos são quadrados de números inteiros não negativos. O número 17 não aparece nessa lista, então a função continua procurando indefinidamente. Já member 16 squares termina porque encontra o valor 16.

como corrigir:
member b [] = False
member b (a:x)
| a == b = True
| a > b = False
| otherwise = member b x

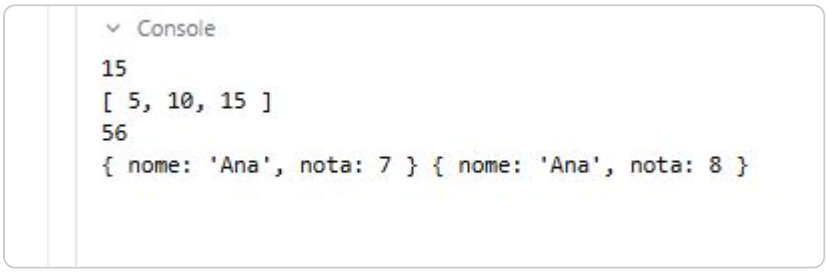
## Prática 6: Você já usa funcional

**Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?**

15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }

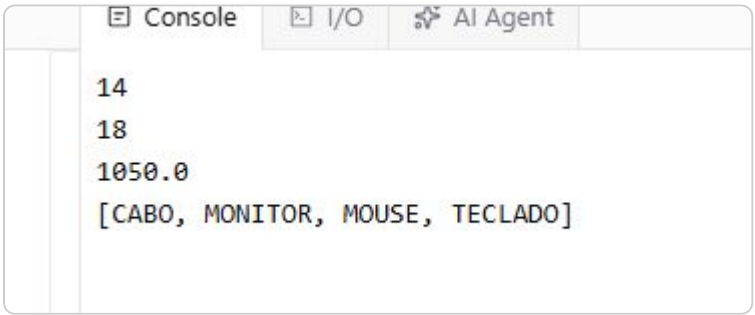
**Saída do JavaScript**

*não respondido*




**Saída do Java**

*não respondido*




**6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?**

Isso acontece por meio de um fechamento (closure). A função devolvida por somador(10) mantém acesso ao ambiente em que foi criada, incluindo o valor de n, mesmo depois que a execução de somador terminou.

**6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?**

f.compose(g).apply(4) executa primeiro g(4), que resulta em 12, e depois f(12), que resulta em 14.
Já f.andThen(g).apply(4) executa primeiro f(4), que resulta em 6, e depois g(6), que resulta em 18.
O método compose corresponde à função compor(f, g) da Prática 1, pois executa primeiro g e depois f.

## Prática 7: O laço vira pipeline

**7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.**

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);

        int soma = nums.stream()
            .filter(n → n % 2 == 0)
            .mapToInt(n → n * n)
            .sum();

        System.out.println(soma);
    }
}
```

**7.2 Segundo exercício escolhido: cole o seu código reescrito.**

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 }
];

const aprovados = alunos
  .filter(aluno ⇒ aluno.nota ⩾ 6)
  .map(aluno ⇒ aluno.nome.toUpperCase());

console.log(aprovados.join(", "));
```

**Saídas das suas versões**

```
1. 244
2. ANA, CARLA, DAVI
```

**7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?**

A versão funcional é mais fácil de ler quando queremos entender as etapas de transformação dos dados, pois cada operação tem uma finalidade específica. Entretanto, a versão com laço pode ser mais fácil de depurar em alguns casos, porque permite acompanhar cada iteração e verificar os valores intermediários com mais facilidade.

## Fechamento: O que fica do paradigma funcional

**Uma função pura pode ler uma variável global que muda.**

Falso

**Em Scheme, a recursão em cauda não faz a pilha crescer.**

Verdadeiro

**O Python otimiza a recursão em cauda.**

Falso

**Em Haskell, [1 ..] só é calculada à medida que é usada.**

Verdadeiro

**filter, map e reduce dispensam laço e variável acumuladora.**

Verdadeiro

**Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.**

Verdadeiro

**8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?**

Considero a versão em Haskell mais fácil de entender neste exemplo, porque apresenta diretamente as operações de gerar a lista, elevar os números ao cubo e somar os resultados. Eu aceitaria um possível custo maior de execução em sistemas nos quais a legibilidade, a manutenção e a facilidade de testes fossem prioridades. Já em sistemas de tempo real ou aplicações que exigem alto desempenho, esse custo precisaria ser avaliado com mais cuidado.

**8.2 Em uma frase: o que mais surpreendeu você nesta atividade?**

O que mais me surpreendeu foi perceber que é possível processar listas e realizar cálculos complexos sem modificar variáveis, utilizando funções e recursão.

**8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?**

O erro está na expressão b <- h. O correto é b <= h, pois a intenção é filtrar os elementos da lista t que são menores ou iguais ao pivô h. O operador <- é utilizado para percorrer uma lista em uma compreensão, enquanto <= faz uma comparação.